# Mudcard

- **IterativeImputer can use many estimators (BayesianRidge, KNN, RandomForest, ExtraTrees, etc.). How should I decide which one to use for my dataset? Should I treat the estimator as a hyperparameter and compare test scores across them, or are there rules of thumb (e.g., dataset size, linear vs. nonlinear relationships, computation time)?**
  - My recommendation is to choose a non-deterministic estimator like a decision tree or a small random forest
  - This allows you to measure the uncertainty due to the imputation
  - If you use linear regression, the imputed values are always the same
  - Computation time can be an important fector too, this is why I wouldn't recommend a random forest with 100 trees for example

# Missing data, part 2

By the end of this lecture, you will be able to

- apply XGBoost to a dataset with missing values
- apply the reduced-features model (also called the pattern submodel approach)
- decide which approach is best for your dataset

## We continue working with the house price data set

- regression problem
- categorical, ordinal, continuous features
- missing data in all feature types

In [1]:
```
# read the data
import pandas as pd
import numpy  as np
from sklearn.model_selection import train_test_split

# Let's load the data
df = pd.read_csv('../data/train.csv')
# drop the ID
df.drop(columns=['Id'],inplace=True)

# the target variable
y = df['SalePrice']
df.drop(columns=['SalePrice'],inplace=True)
# the unprocessed feature matrix
X = df.values
```

```
print(X.shape)
# the feature names
ftrs = df.columns
```

```
(1460, 79)
```

In [2]:
```
perc_missing_per_ftr = df.isnull().sum(axis=0)/df.shape[0]
print('fraction of missing values in features:')
print(perc_missing_per_ftr[perc_missing_per_ftr > 0])
print('data types of the features with missing values:')
print(df[perc_missing_per_ftr[perc_missing_per_ftr > 0].index].dtypes)
frac_missing = sum(df.isnull().sum(axis=1)!=0)/df.shape[0]
print('fraction of points with missing values:',frac_missing)
```

```
fraction of missing values in features:
LotFrontage     0.177397
Alley           0.937671
MasVnrType      0.597260
MasVnrArea      0.005479
BsmtQual        0.025342
BsmtCond        0.025342
BsmtExposure    0.026027
BsmtFinType1    0.025342
BsmtFinType2    0.026027
Electrical      0.000685
FireplaceQu     0.472603
GarageType      0.055479
GarageYrBlt     0.055479
GarageFinish    0.055479
GarageQual      0.055479
GarageCond      0.055479
PoolQC          0.995205
Fence           0.807534
MiscFeature     0.963014
dtype: float64
data types of the features with missing values:
LotFrontage     float64
Alley               str
MasVnrType          str
MasVnrArea      float64
BsmtQual            str
BsmtCond            str
BsmtExposure        str
BsmtFinType1        str
BsmtFinType2        str
Electrical          str
FireplaceQu         str
GarageType          str
GarageYrBlt     float64
GarageFinish        str
GarageQual          str
GarageCond          str
PoolQC              str
Fence               str
MiscFeature         str
dtype: object
fraction of points with missing values: 1.0
```

In [3]:
```
# let's split to train, CV, and test
X_other, X_test, y_other, y_test = train_test_split(df, y, test_size=0.2, ra
X_train, X_CV, y_train, y_CV = train_test_split(X_other, y_other, test_size=

print(X_train.shape)
print(X_CV.shape)
print(X_test.shape)
```

```
(876, 79)
(292, 79)
(292, 79)
```

In [4]:
```
# collect the various features
cat_ftrs = ['MSZoning','Street','Alley','LandContour','LotConfig','Neighborh
            'BldgType','HouseStyle','RoofStyle','RoofMatl','Exterior1st','Ex
           'Heating','CentralAir','Electrical','GarageType','PavedDrive','Mi
ordinal_ftrs = ['LotShape','Utilities','LandSlope','ExterQual','ExterCond','
               'BsmtFinType1','BsmtFinType2','HeatingQC','KitchenQual','Func
               'GarageQual','GarageCond','PoolQC','Fence']
ordinal_cats = [['Reg','IR1','IR2','IR3'],['AllPub','NoSewr','NoSeWa','ELO']
               ['Po','Fa','TA','Gd','Ex'],['Po','Fa','TA','Gd','Ex'],['NA','
               ['NA','Po','Fa','TA','Gd','Ex'],['NA','No','Mn','Av','Gd'],['
               ['NA','Unf','LwQ','Rec','BLQ','ALQ','GLQ'],['Po','Fa','TA','G
               ['Sal','Sev','Maj2','Maj1','Mod','Min2','Min1','Typ'],['NA','
               ['NA','Unf','RFn','Fin'],['NA','Po','Fa','TA','Gd','Ex'],['NA
               ['NA','Fa','TA','Gd','Ex'],['NA','MnWw','GdWo','MnPrv','GdPrv
num_ftrs = ['MSSubClass','LotFrontage','LotArea','OverallQual','OverallCond'
            'MasVnrArea','BsmtFinSF1','BsmtFinSF2','BsmtUnfSF','TotalBsmtSF
            'LowQualFinSF','GrLivArea','BsmtFullBath','BsmtHalfBath','FullB
            'KitchenAbvGr','TotRmsAbvGrd','Fireplaces','GarageYrBlt','Garag
            'OpenPorchSF','EnclosedPorch','3SsnPorch','ScreenPorch','PoolAr
```

In [5]:
```
# preprocess with pipeline and columntransformer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
import warnings
warnings.filterwarnings("ignore")

# one-hot encoder
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant',fill_value='missing')),
    ('onehot', OneHotEncoder(sparse_output=False,handle_unknown='ignore'))])

# ordinal encoder
ordinal_transformer = Pipeline(steps=[
    ('imputer2', SimpleImputer(strategy='constant',fill_value='NA')),
    ('ordinal', OrdinalEncoder(categories = ordinal_cats))])

# standard scaler
numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())])

# collect the transformers
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, num_ftrs),
        ('cat', categorical_transformer, cat_ftrs),
        ('ord', ordinal_transformer, ordinal_ftrs)])
```

In [6]:
```
# fit_transform the training set
X_prep = preprocessor.fit_transform(X_train)
# collect feature names
feature_names = preprocessor.get_feature_names_out()
```

```
df_train = pd.DataFrame(data=X_prep,columns=feature_names)
print(df_train.shape)

# transform the CV
df_CV = preprocessor.transform(X_CV)
df_CV = pd.DataFrame(data=df_CV,columns = feature_names)
print(df_CV.shape)

# transform the test
df_test = preprocessor.transform(X_test)
df_test = pd.DataFrame(data=df_test,columns = feature_names)
print(df_test.shape)
```

```
(876, 220)
(292, 220)
(292, 220)
```

In [7]:
```
print('data dimensions:',df_train.shape)
perc_missing_per_ftr = df_train.isnull().sum(axis=0)/df_train.shape[0]
print('fraction of missing values in features:')
print(perc_missing_per_ftr[perc_missing_per_ftr > 0])
frac_missing = sum(df_train.isnull().sum(axis=1)!=0)/df_train.shape[0]
print('fraction of points with missing values:',frac_missing)
```

```
data dimensions: (876, 220)
fraction of missing values in features:
num__LotFrontage    0.173516
num__MasVnrArea     0.004566
num__GarageYrBlt    0.050228
dtype: float64
fraction of points with missing values: 0.2237442922374429
```

# Missing data, part 2

By the end of this lecture, you will be able to

- **apply XGBoost to a dataset with missing values**
- apply the reduced-features model (also called the pattern submodel approach)
- decide which approach is best for your dataset

## XGBoost

- eXtreme Gradient Boosting - a popular tree-based method
- blog post and paper
- more advanced than random forest
  - it has l1 and l2 regularization while random forest does not
  - trees are not independent
    - the next tree is built to improve the previous tree
    - less trees are necessary to achieve same accuracy

    - but XGBoost trees can overfit
  - handles missing values well

## XGBoost and missing values

- some sklearn models raise an error if the feature matrix (X) contains nans - but see here
- XGBoost doesn't!
- If a feature with missing values is split:
  - XGBoost tries to put the points with missing values to the left and right
  - calculates a metric called gain for both options
  - puts the points with missing values to the side with the higher gain
- if missingness correlates with the target variable, XGBoost extracts this info!

In [8]:
```
import xgboost
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

param_grid = {"learning_rate": [0.03],
              "n_estimators": [10000],
              "seed": [0],
              #"reg_alpha": [0e0, 1e-2, 1e-1, 1e0, 1e1, 1e2],
              #"reg_lambda": [0e0, 1e-2, 1e-1, 1e0, 1e1, 1e2],
              "missing": [np.nan],
              #"max_depth": [1,3,10,30,100],
              "colsample_bytree": [0.9],
              "subsample": [0.66]}

XGB = xgboost.XGBRegressor()
XGB.set_params(**ParameterGrid(param_grid)[0],early_stopping_rounds=50) # ON
XGB.fit(df_train,y_train,eval_set=[(df_CV, y_CV)], verbose=False)
y_CV_pred = XGB.predict(df_CV)
print('the CV RMSE:',np.sqrt(mean_squared_error(y_CV,y_CV_pred)))
y_test_pred = XGB.predict(df_test)
print('the test RMSE:',np.sqrt(mean_squared_error(y_test,y_test_pred)))
print('the test R2:',r2_score(y_test,y_test_pred))

print(XGB.best_iteration)
```

```
the CV RMSE: 24584.178326720623
the test RMSE: 33324.32624975335
the test R2: 0.8391927480697632
333
```

### XGB notes

- do not tune the number of trees, set it to a very large value like 10000
- use early stopping instead! it will automatically determine the best number of trees based on a validation set.

- there are a large number of hyperparameters in XGB
  - tune maybe `max_depth`, `reg_alpha`, and `reg_lambda`
- the default values of some hyperparameters are not optimal
  - set `colsample_bytree` and `subsample` to values a bit smaller than 1 to avoid overfitting

## Quiz 1

# Missing data, part 2

By the end of this lecture, you will be able to

- apply XGBoost to a dataset with missing values
- **apply the reduced-features model (also called the pattern submodel approach)**
- decide which approach is best for your dataset

## Reduced-features model (or pattern submodel approach)

- first described in 2007 in a JMLR article as the reduced features model
- in 2018, "rediscovered" as the pattern submodel approach in Biostatistics

My test set:

| index | feature 1 | feature 2 | feature 3 | target var |
|---|---|---|---|---|
| 0 | NA | 45 | NA | 0 |
| 1 | NA | NA | 8 | 1 |
| 2 | 12 | 6 | 34 | 0 |
| 3 | 1 | 89 | NA | 0 |
| 4 | 0 | NA | 47 | 1 |
| 5 | 687 | 24 | 67 | 1 |
| 6 | NA | 23 | NA | 1 |

To predict points 0 and 6, I will use train and CV points that are complete in feature 2 (and other complete features not included here).

To predict point 1, I will use train and CV points that are complete in feature 3 (and other complete features not included here).

To predict point 2 and 5, I will use train and CV points that are complete in features 1-3 (and other complete features not included here).

Etc. We will train as many models as the number of patterns in test/deployment.

# How to determine the patterns?

In [9]:
```
mask = df_test[['num__LotFrontage','num__MasVnrArea','num__GarageYrBlt']].is

unique_rows, counts = np.unique(mask, axis=0,return_counts=True)
print(unique_rows.shape) # 6 patterns, we will train 6 models
for i in range(len(counts)):
    print(unique_rows[i],counts[i])
```

```
(6, 3)
[False False False] 223
[False False  True] 21
[False  True False] 1
[ True False False] 44
[ True False  True] 2
[ True  True False] 1
```

In [10]:
```
import xgboost
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

def xgb_model(X_train, Y_train, X_CV, y_CV, X_test, y_test, verbose=1):

    # make into row vectors to avoid an sklearn/xgb warning
    Y_train = np.reshape(np.array(Y_train), (1, -1)).ravel()
    y_CV = np.reshape(np.array(y_CV), (1, -1)).ravel()
    y_test = np.reshape(np.array(y_test), (1, -1)).ravel()

    XGB = xgboost.XGBRegressor(n_jobs=1)

    # find the best parameter set
    param_grid = {"learning_rate": [0.03],
                  "n_estimators": [10000],
                  "seed": [0],
                  #"reg_alpha": [0e0, 1e-2, 1e-1, 1e0, 1e1, 1e2],
                  #"reg_lambda": [0e0, 1e-2, 1e-1, 1e0, 1e1, 1e2],
                  "missing": [np.nan],
                  #"max_depth": [1,3,10,30,100,],
                  "colsample_bytree": [0.9],
                  "subsample": [0.66]}

    pg = ParameterGrid(param_grid)

    scores = np.zeros(len(pg))

    for i in range(len(pg)):
        if verbose >= 5:
            print("Param set " + str(i + 1) + " / " + str(len(pg)))
        params = pg[i]
        XGB.set_params(**params,early_stopping_rounds=50)
        eval_set = [(X_CV, y_CV)]
```

```
        XGB.fit(X_train, Y_train,
                eval_set=eval_set, verbose=False)# with early stopping
        y_CV_pred = XGB.predict(X_CV)
        scores[i] = mean_squared_error(y_CV,y_CV_pred)

    best_params = np.array(pg)[scores == np.max(scores)]
    if verbose >= 4:
        print('Test set max score and best parameters are:')
        print(np.max(scores))
        print(best_params)

    # test the model on the test set with best parameter set
    XGB.set_params(**best_params[0],early_stopping_rounds=50)
    XGB.fit(X_train,Y_train,
            eval_set=eval_set, verbose=False)
    y_test_pred = XGB.predict(X_test)

    if verbose >= 1:
        print ('The MSE is:',mean_squared_error(y_test,y_test_pred))
    if verbose >= 2:
        print ('The predictions are:')
        print (y_test_pred)
    if verbose >= 3:
        print("Feature importances:")
        print(XGB.feature_importances_)

    return (mean_squared_error(y_test,y_test_pred), y_test_pred, XGB.feature

# Function: Reduced-feature XGB model
# all the inputs need to be pandas DataFrames
def reduced_feature_xgb(X_train, Y_train, X_CV, y_CV, X_test, y_test):

    # find all unique patterns of missing value in test set
    mask = X_test.isnull()
    unique_rows = np.array(np.unique(mask, axis=0))
    all_y_test_pred = pd.DataFrame()

    print('there are', len(unique_rows), 'unique missing value patterns.')

    # divide test sets into subgroups according to the unique patterns
    for i in range(len(unique_rows)):
        print ('working on unique pattern', i)
        ## generate X_test subset that matches the unique pattern i
        sub_X_test = pd.DataFrame()
        sub_y_test = pd.Series(dtype=float)
        for j in range(len(mask)): # check each row in mask
            row_mask = np.array(mask.iloc[j])
            if np.array_equal(row_mask, unique_rows[i]): # if the pattern ma

                sub_X_test = pd.concat([sub_X_test,X_test.iloc[[j]]])# apper
                sub_y_test = pd.concat([sub_y_test, y_test.iloc[[j]]])# appe

        sub_X_test = sub_X_test[X_test.columns[~unique_rows[i]]]

        ## choose the corresponding reduced features for subgroups
        sub_X_train = pd.DataFrame()
```

```
        sub_Y_train = pd.DataFrame()
        sub_X_CV = pd.DataFrame()
        sub_y_CV = pd.DataFrame()
        # 1. remove the feature columns that have nans in the corresponding
        sub_X_train = X_train[X_train.columns[~unique_rows[i]]]
        sub_X_CV = X_CV[X_CV.columns[~unique_rows[i]]]
        # 2.remove the rows in the sub_X_train and sub_X_CV that have any na
        sub_X_train = sub_X_train.dropna()
        sub_X_CV = sub_X_CV.dropna()
        # 3.remove the sub_Y_train and sub_y_CV accordingly
        sub_Y_train = Y_train.iloc[sub_X_train.index]
        sub_y_CV = y_CV.iloc[sub_X_CV.index]

        # run XGB
        sub_y_test_pred = xgb_model(sub_X_train, sub_Y_train, sub_X_CV,
                                    sub_y_CV, sub_X_test, sub_y_test, ver
        sub_y_test_pred = pd.DataFrame(sub_y_test_pred[1],columns=['sub_y_te
                                       index=sub_y_test.index)
        print('   RMSE:',np.sqrt(mean_squared_error(sub_y_test,sub_y_test_pr
        # collect the test predictions
        all_y_test_pred = pd.concat([all_y_test_pred, sub_y_test_pred])

    # rank the final y_test_pred according to original y_test index
    all_y_test_pred = all_y_test_pred.sort_index()
    y_test = y_test.sort_index()

    # get global RMSE
    total_RMSE = np.sqrt(mean_squared_error(y_test,all_y_test_pred))
    total_R2 =  r2_score(y_test,all_y_test_pred)
    return total_RMSE, total_R2
```

In [11]:
```
RMSE, R2 = reduced_feature_xgb(df_train, y_train, df_CV, y_CV, df_test, y_te
print('final RMSE:', RMSE)
print('final R2:', R2)
```

```
there are 6 unique missing value patterns.
working on unique pattern 0
   RMSE: 36632.93897071807
working on unique pattern 1
   RMSE: 12720.122430512318
working on unique pattern 2
   RMSE: 6195.65625
working on unique pattern 3
   RMSE: 20333.757172762893
working on unique pattern 4
   RMSE: 22206.201991008696
working on unique pattern 5
   RMSE: 49318.03125
final RMSE: 33326.26362943341
final R2: 0.8391740548538331
```

# Quiz 2

# Missing data, part 2

By the end of this lecture, you will be able to

- apply XGBoost to a dataset with missing values
- apply the reduced-features model (also called the pattern submodel approach)
- **decide which approach is best for your dataset**

## Which approach is best for my data?

- **XGB**: run $n$ XGB models with $n$ different seeds
- **imputation**: prepare $n$ different imputations and run $n$ models on them
- **reduced-features**: run $n$ reduced-features model with $n$ different seeds
- rank the three methods based on how significantly different the corresponding mean scores are

# Mudcard

In [ ]: